

The compiler writes a destructor for you and it destroys the **members**. It destroys the pointer. It does not touch what the pointer points at.

If your class calls `new`, your class defines a destructor.

Re-run the sanitizer. Clean.

Beat 4. The debugger, on our own class (16 min)

Nobody has watched anyone drive it; it arrived in writing only, in the wk2 lab and inside a03. a05 §9 wants a breakpoint on `delete[]` and an address read out of Variables. Drive `PyList`; leave a05's `experiment` for them to meet cold.

1. **Breakpoint.** Gutter left of the line number, red dot, on `int cap = ...`.
2. **F5**, pick the configuration. It stops on the dot.
3. **Variables.** Expand `this`; add `myArray` to **Watch**.
4. **F10** over, **F11** into, **F5** continue.

Append nine items, and predict before each **F5**. Will it stop again? Will the address change?

Monday's version stops on every `append`, with a new address every time. **Today's** stops five times, capacity 1, 2, 4, 8, 16, and the stops get rarer as the array grows. The array moves, and anything holding the old address is holding a lie.

Close: Friday, these same four moves on a program that misbehaves.

Beat 5. What this unlocks (6 min)

- **a05 TODO 5, `append`**. Written twice now; `setSize` in `Trace.cpp` is the same four steps, worked out in full.
- **TODO 6, the destructor**. One line, and beat 3 is the reason.
- That is §7. **§9 comes after it, and in order**. Friday: the copy that bites.
- **If asked:** a05 checks `i >= mySize` alone, because an unsigned index cannot be negative.

CS112: Wednesday 30 September, 12:15 to 1:20

Monday we wrote `PyList` together. A class holding `Item *myArray`, `mySize` and `myCapacity`; a constructor that sets the pointer to `nullptr`; an `append` that makes room for one more item every time it is called. It compiles, it passes its test, and every line in it is correct.

Today: the object is not the data. `myArray` holds an address, the items sit somewhere else, and that address moves every time the array grows. Two things fall out of seeing that. Our `append` is doing far more work than it needs to, and nothing in the class ever gives the memory back.

TIME	BEAT
12:15	Daily quiz
12:20	1. What does our <code>append</code> cost?
12:32	2. Make it cost less, and ? :
12:48	3. It leaks right now
12:58	4. The debugger, on our own class
13:14	5. What this unlocks

If the clock slips, shorten beat 1. Beat 4 is the one nothing else replaces.

Beat 1. What does our `append` cost? (12 min)

Open `PyList.cpp`. Run `make test`. It passes, as expected. Nothing here is broken. Now let's think about what these operations cost.

Talk to your neighbor. Every `append` asks for a new array and copies everything across. How many item copies is that, for 100 appends?

Collect answers before confirming anything. The sum is $1 + 2 + \dots + 99$, worth doing on the board because the shape is the point: the cost grows with the square of the number of appends. Measured: **4,950 item copies and 100 allocations** for those 100 appends.

Then hand the problem back rather than the answer:

Second question, same neighbor. The copying is what costs. How could we copy less often?

Whatever comes back, cost it on the board. Two answers are likely:

STRATEGY	100 APPENDS	1000 APPENDS
one more slot each time	4,950 copies	499,500 copies
ten more slots each time	450 copies	49,500 copies
twice the room each time	127 copies	1,023 copies

"Ten extra" is a real improvement and still the wrong shape: ten times the appends, a hundred times the copying. Only doubling's second column barely moves. If nobody reaches it, ask what happens when the extra room is proportional to what we already have instead of a fixed number. Week 5 names why.

Beat 2. Make it cost less, and `? :` (16 min)

Write it together, starting from the condition. Ask when the array needs to grow, and take `mySize == myCapacity` from the room.

```
void PyList::append(Item it) {
    if (mySize == myCapacity) {        // full; empty is full too
        int cap = (myCapacity == 0) ? 1 : myCapacity * 2;
        Item *bigger = new Item[cap];
        for (int i = 0; i < mySize; ++i)
            bigger[i] = myArray[i];
        delete [] myArray;
        myArray = bigger;
        myCapacity = cap;
    }
    myArray[mySize] = it;
    ++mySize;
}
```

Monday's `if (mySize == 0) ... else ...` is gone, and that is worth stopping on: **the empty list was never a special case**. It was an array with no room left, which the one condition already catches.

The `? :` line is the only unfamiliar thing on the screen. Write what they can already read first, then put the one line beside it and read it aloud as **"if this, then that, otherwise the other"**.

```
int cap;
if (myCapacity == 0) cap = 1;
else cap = myCapacity * 2;
```

`if` is a **statement**: it *does* something. `? :` is an **expression**: it *is* a value. That is why it can sit on the right of an `=` and the `if` cannot.

Three rules: choose **between two values for one variable**, never nest them, no side effects inside. Then the trap, worth running live:

```
cout << x > 0 ? "positive" : "negative";
```

```
error: no match for 'operator>' ('std::basic_ostream<char>' and
'int')
```

`<<` binds tighter, so the compiler read `(cout << x) > 0`. Parenthesise it inside any larger expression.

Re-run Monday's test. It still passes: after two appends the capacity is still 2, because doubling goes 0, 1, 2. **Our test cannot tell the two versions apart.** Worth asking the room what a passing test does and does not prove.

Beat 3. It leaks right now (10 min)

Question for the room. A `PyList` goes out of scope at the end of a function. What happens to the array it asked for?

Settle it by running it, not by saying it.

```
g++ -std=c++17 -g -fsanitize=address -o leak leak.cpp PyList.cpp
&& ./leak
```

```
SUMMARY: AddressSanitizer: 20 byte(s) leaked in 1 allocation(s).
```

```
PyList::~PyList() { delete [] myArray; myArray = nullptr; }
```

Ask where the missing line goes before writing it. Then: